Kubernetes & HA Chat

High Availability (HA) Clusters — Explained Simply

What it means:

A “high availability cluster” is a group of servers (nodes) working together so that if one server fails, another one automatically takes over — and users barely notice any downtime.

Think of it like a relay race team where if one runner suddenly can’t continue, another runner who’s already warmed up jumps in immediately, so the race doesn’t stop.

Why it matters:

Normally, if you have one server running your app or database, and that server crashes, your service goes down until someone fixes it. In an HA cluster, there are multiple servers ready to pick up the work instantly, so your service stays “available” — hence the name.

How to Make a Cluster Highly Available

Here are the core building blocks, in plain terms:

1. Redundancy (no single point of failure)

Have at least 2-3 copies of everything important — multiple servers, multiple network paths, multiple power supplies. If one thing breaks, there’s a backup ready.

2. Load Balancer

Put a “traffic director” in front of your servers. It sends user requests to whichever server is healthy and available. If one server dies, it stops sending traffic there.

3. Heartbeat / Health Checks

Servers constantly “ping” each other (like a heartbeat) to say “I’m alive!” If a server stops responding, the others notice within seconds and react.

4. Failover Mechanism

When a server fails, another one automatically takes over its job — this is called “failover.” This can be: - **Active-Passive**: One server does the work, a backup sits idle ready to jump in. - **Active-Active**: Multiple servers share the work simultaneously; if one drops, the rest absorb the load.

5. Shared or Replicated Storage

Data needs to be accessible no matter which server is “in charge.” This is done via: - Shared storage (like a SAN) both servers can access, or - Data replication (data copied in real-time across servers).

6. Quorum / Split-Brain Prevention

When servers lose contact with each other, you need a rule to decide who’s “in charge” to avoid two servers both thinking they’re the primary (this is called split-brain and causes data corruption). Usually done with a voting system — majority wins.

7. Geographic Redundancy (optional, for extra safety)

Spread servers across different data centers or regions, so even a natural disaster or power outage

at one location doesn't take everything down.

Simple Analogy

Imagine a hospital emergency room: - **Multiple doctors on duty** = redundancy - **A triage nurse directing patients** = load balancer - **Doctors checking in with each other** = heartbeat/health checks - **Another doctor stepping in if one goes on break** = failover - **Shared patient records everyone can access** = shared storage - **A clear chain of command to avoid confusion** = quorum

---

In short: High availability = redundancy + automatic failover + health monitoring + no single point of failure. The goal is that if any *one* thing breaks, the system keeps running without anyone noticing.

Cordon, Drain, and Upgrading/Downgrading Nodes — Explained Simply

This is mostly a **Kubernetes** concept (though similar ideas exist in other cluster systems). Let's break it down.

The Problem This Solves

Imagine you need to fix, upgrade, or reboot one server (node) in your cluster. But that server is currently running live applications for users. If you just yank it offline, you crash whatever was running on it. **Cordon and drain** let you safely empty a server before touching it.

---

1. Cordon — "Stop sending new work here"

Cordoning marks a node as **unschedulable**. It's like putting a "Do Not Enter" sign on a server.

- Existing apps keep running on it — nothing is kicked off yet.
- But no *new* apps will be placed on that node going forward.

Command:

```
kubectl cordon node-1
```

Analogy: Think of a restaurant putting a "this section is closing soon" sign on a table. Current customers can keep eating, but no new customers get seated there.

---

2. Drain — "Evacuate everything from here"

Draining takes it a step further — it safely evicts all the running apps (pods) off the node and reschedules them onto other healthy nodes.

- It automatically cordons the node first.
- Then it gracefully moves the workloads elsewhere, respecting things like "don't take down all copies of this app at once."

Command:

```
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data
```

Analogy: Now the restaurant tells the remaining customers at that table, "please move to another table," so they can close this one down completely for cleaning.

Once draining is done, the node has **zero workloads** running on it — it's safe to reboot, upgrade,

or remove.

---

3. Upgrade (or Downgrade) the Node

Now that the node is empty and safe:

- **Upgrade**: Update the OS, Kubernetes version (kubelet, kube-proxy, etc.), patch software, or apply security updates.
- **Downgrade**: Roll back to a previous version if the upgrade caused issues.

Typical flow for a rolling upgrade across a whole cluster:

```
For each node, one at a time:
 1. Cordon the node        → stop new work from landing here
 2. Drain the node         → move existing work elsewhere
 3. Upgrade/downgrade it   → patch OS, update Kubernetes version, etc.
 4. Uncordon the node      → allow new work again
 5. Move to the next node
```

4. Uncordon — “Reopen for business”

Once the node is fixed and healthy again:

```
kubectl uncordon node-1
```

This removes the “Do Not Enter” sign, so the scheduler can start placing new workloads on it again.

Why Do It One Node at a Time?

This is exactly how **high availability** ties back in — since you have redundancy (multiple nodes/replicas), you can safely take *one* node down without downtime, because the others absorb the load. If you tried to upgrade all nodes simultaneously, you’d cause an outage.

Quick Summary Table

Step	Purpose	Command
Cordon	Block new workloads	`kubectl cordon <node>`
Drain	Evict existing workloads safely	`kubectl drain <node>`
Upgrade/Downgrade	Patch or change version	(OS/tool specific)
Uncordon	Allow scheduling again	`kubectl uncordon <node>`

Analogy recap: - **Cordon** = “No new customers, but current ones can stay” - **Drain** = “Everyone out, we’re closing this section” - **Upgrade/downgrade** = “Renovate the section” - **Uncordon** = “Reopen for business”

HPA and VPA — Explained Simply

Both are Kubernetes tools that **automatically adjust resources** so your apps run smoothly without you manually intervening every time traffic changes. The difference is *what* they scale.

The Core Difference (in one line)

- **HPA (Horizontal Pod Autoscaler)** = adds/removes **copies (pods)** of your app
- **VPA (Vertical Pod Autoscaler)** = adds/removes **power (CPU/memory)** to each existing pod

Think of it like a coffee shop: - **HPA** = opening more cash registers when the line gets long (more staff/counters) - **VPA** = giving your one existing cashier a faster register and more counter space

(upgrading the resource)

---

1. HPA — Horizontal Pod Autoscaler

What it does

Watches metrics like CPU usage, memory usage, or custom metrics (like requests per second), and automatically **increases or decreases the number of pod replicas** running your application.

How it works

1. You set a target — e.g., "keep average CPU usage at 50% per pod."
2. HPA continuously checks real usage (every ~15 seconds by default).
3. If usage goes above target → it **spins up more pods**.
4. If usage drops → it **scales pods down**.

Example

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

This says: "Keep CPU at 50% per pod, but never go below 2 pods or above 10."

When to use HPA

- Your app is **stateless** and can easily run multiple copies (web servers, APIs, microservices).
- Traffic is unpredictable or spikes (e.g., Black Friday sales, viral content).

Analogy

It's like a call center. When call volume spikes, you bring in more agents. When it's quiet, you send some agents home. Each agent (pod) has the same skill level (same resource size) — you just change *how many* you have.

---

2. VPA — Vertical Pod Autoscaler

What it does

Watches how much CPU/memory a pod *actually* uses over time, and automatically **adjusts the resource requests/limits** for that pod — giving it more or less CPU/memory as needed.

How it works

1. VPA has a "recommender" that watches historical usage.
2. It calculates the ideal CPU/memory a pod should have.
3. Depending on mode, it can:
    - Just **recommend** values (you apply manually), or

- **Automatically update** the pod (which requires restarting/recreating the pod with new resource settings — pods can't be resized live in most versions).

Example

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Auto"
```

This says: "Watch my-app's resource usage and automatically adjust CPU/memory requests as needed."

Update Modes

Mode	What it does
`Off`	Just gives recommendations, doesn't change anything
`Initial`	Only sets resources when pod is first created
`Recreate`	Actually restarts pods with new resource values
`Auto`	Same as Recreate currently (will support live resize in future)

When to use VPA

- Your app is **stateful** or can't easily run multiple replicas (like a database, or a legacy monolith).
- You don't know the "right" amount of CPU/memory to assign and want Kubernetes to learn and adjust it for you.
- Avoiding **over-provisioning** (wasting money on resources you don't use) or **under-provisioning** (causing crashes/throttling).

Analogy

Think of it like a food delivery driver. Instead of hiring more drivers (HPA), you give your one driver a bigger vehicle when they need to carry more packages, or downgrade to a smaller car when demand is low.

Important Gotcha: HPA and VPA Can Conflict!

If you use **both together on CPU/memory metrics**, they can fight each other: - VPA changes pod size → this changes CPU/memory usage % → HPA sees changed metrics → tries to scale pod count → creates confusing feedback loops.

Safe way to combine them: - Use **VPA on CPU/memory** - Use **HPA on a different, custom metric** — like requests-per-second, queue length, or custom app-level metrics (not CPU/memory)

This way they don't step on each other's toes.

Quick Comparison Table

Feature	HPA	VPA
What it scales	Number of pod replicas	CPU/memory per pod
Best for	Stateless apps, microservices	Stateful apps, databases, unpredictable resource needs

Restart required?	No (new pods just get added)	Yes (existing pod needs recreation to resize)
Good for traffic spikes	Yes	Not really (slower reaction, requires restart)
Good for “right-sizing” resources	Not directly	Yes
Can run together	Careful — avoid same metric conflicts	Same

Simple Summary

- **HPA = more copies** (scale out/in) — best for handling variable traffic
- **VPA = bigger/smaller copies** (scale up/down) — best for right-sizing resource usage

Network Policies in Kubernetes — Explained Simply

By default in Kubernetes, **every pod can talk to every other pod** — no restrictions at all. That’s like an office building where any employee can walk into any room, including the server room, HR files, or the CEO’s office. **Network Policies** let you add locks on doors — controlling exactly which pods can talk to which.

The Core Idea

A **Network Policy** is a set of rules that controls: - **Ingress** — who is allowed to **send traffic INTO** a pod - **Egress** — who is allowed to **receive traffic FROM (going out of)** a pod

Think of it like a bouncer at a club checking: - Who’s allowed to **come in** (ingress) - Who’s allowed to **leave and go where** (egress)

Important Default Behavior

Without any network policy — all pods can freely communicate with all other pods, and with the outside world. It’s a totally open network.

Once you apply a Network Policy to a pod (using a `podSelector`), that pod becomes “isolated” — it now **only** allows what the policy explicitly permits. Everything else gets **blocked** by default (implicit deny).

Key point: Network Policies are **whitelist-based**, not blacklist-based. You can’t say “block this one specific pod” — you say “only allow these specific things,” and everything else is automatically denied.

Basic Structure of a Network Policy

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: my-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
```

```
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
```

Breaking this down:

- `podSelector` → **Which pods does this policy apply to?** (here: pods labeled `app: backend`)
- `policyTypes` → Does this control Ingress, Egress, or both?
- `ingress` → **Who can send traffic IN** to these pods? (here: only pods labeled `app: frontend`, and only on port 8080)
- `egress` → **Where can these pods send traffic OUT to?** (here: only pods labeled `app: database`, on port 5432)

In plain English: "Only the frontend pod can talk to the backend pod (on port 8080), and the backend pod can only talk to the database pod (on port 5432). Everything else is blocked."

Common Selectors (Who the rule applies to)

Selector	Meaning
`podSelector`	Match pods by label, within the same namespace
`namespaceSelector`	Match all pods within specific namespaces
`ipBlock`	Match by IP address range (CIDR) — useful for external traffic

Example: allow traffic from a specific namespace

```
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          team: analytics
```

This means: "Allow traffic from any pod in namespaces labeled `team: analytics`."

Example: allow traffic only from a specific IP range

```
  ingress:
  - from:
    - ipBlock:
        cidr: 203.0.113.0/24
```

This means: "Only allow traffic from this external IP range."

Common Real-World Examples

1. Deny all traffic to a pod (lockdown mode)

```
  apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: deny-all
```

```yaml
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

This applies to **all pods** (empty selector = all) and allows **no ingress rules** = blocks all incoming traffic.

Analogy: Locking every door in the building with no keys given out.

---

2. Allow traffic only from within the same namespace

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-same-namespace
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector: {}
```

This means: “Pods can only be reached by other pods in the same namespace.” (Empty `podSelector` under `from` = any pod in same namespace)

---

3. Allow only frontend pods to reach backend pods

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

Analogy: Only people wearing a “Frontend Team” badge can enter the “Backend Room.”

---

4. Allow egress only to the internet (block internal pod-to-pod)

```yaml
egress:
- to:
  - ipBlock:
      cidr: 0.0.0.0/0
      except:
      - 10.0.0.0/8   # exclude internal cluster IP range
```

This means: “This pod can talk to the outside internet, but not to other internal pods.”

Important Gotchas

1. You need a CNI plugin that supports Network Policies

Kubernetes itself doesn’t enforce these rules — it’s the underlying **network plugin (CNI)** that does. Not all CNIs support it!

CNI Plugin	Supports Network Policies?
Calico	Yes

Cilium	Yes (also has enhanced policies)
Weave Net	Yes
Flannel	No (by default — needs Calico add-on)

If your CNI doesn't support it, **your network policies do nothing** — silently. This trips people up a lot.

2. Policies are additive, not overriding

If you have multiple policies targeting the same pod, Kubernetes **combines them** — allowing the union of everything permitted across all matching policies. You can't write a policy that explicitly overrides another to deny something.

3. Once isolated, always isolated (for that direction)

If a pod is selected by ANY policy with `Ingress` in `policyTypes`, it becomes ingress-isolated — even if that specific policy doesn't mention ingress rules for it. This can accidentally lock pods down more than intended if you're not careful.

4. Default allow, unless selected

Pods not selected by any Network Policy remain completely open. You must intentionally apply policies to pods you want to restrict.

Real-World Best Practice Pattern

A common, secure setup:

```
1. Apply a "deny-all" policy to the whole namespace (locks everything down)
2. Add specific policies to allow only necessary connections:
   - Frontend → Backend
   - Backend → Database
   - Everything → DNS (kube-dns/CoreDNS, port 53) — often forgotten!
   - Everything → Metrics/monitoring scraper (if using Prometheus)
```

Important: People often forget to allow **DNS traffic** (UDP/TCP port 53) after locking things down — this breaks service discovery and causes confusing failures where pods can't resolve service names.

```yaml
egress:
- to:
  - namespaceSelector: {}
  ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
```

Quick Analogy Recap

Kubernetes Concept	Analogy
No Network Policy	Office with no locks — anyone can enter any room
`podSelector`	Which room this lock applies to
`ingress`	Who can enter the room
`egress`	Where the room's occupants are allowed to go
`namespaceSelector`	Allow entry to anyone with a badge from a certain department
`ipBlock`	Allow entry only from a

`ipBlock`	specific street address (external IP range)
Missing CNI support	You installed a lock, but the door has no strike plate — it doesn’t actually work

---

Summary

- **Default:** All pods can talk to all pods (no restrictions).
- **Network Policies = whitelist firewalls** for pod-to-pod (and external) traffic.
- Define **ingress** (incoming) and **egress** (outgoing) rules using **podSelector**, **namespaceSelector**, or **ipBlock**.
- Requires a **CNI plugin that supports it** (Calico, Cilium, Weave — not plain Flannel).
- Best practice: **deny-all by default**, then **explicitly allow** only what’s needed — and don’t forget DNS!